



Κατανεμημένα Συστήματα

Εργαστηριακή Άσκηση 2

ΣΤ3-Α – Πέμπτη 14:00 - 16:00

Ρόμπερτ Πολοβίνα - 23390338

Εξάμηνο 8ο

2025-2026

Μέρος Α:

Δημιουργία κλάσης κρατήσεων:

Ξεκινάμε την υλοποίηση του RMI προγράμματος με την κλάση με την οποία θα δημιουργούμε τα αντικείμενα κρατήσεων που θα περιέχουν όλα τα σχετικά δεδομένα. Συγκεκριμένα θέλουμε την κατάσταση της κράτησης (επιτυχία/αποτυχία), τον αριθμό των διαθέσιμων δωματίων, αν ο χρήστης επιθυμεί κράτηση για λιγότερα διαθέσιμα δωμάτια, τον αριθμό των δωματίων για τα οποία έγινε κράτηση, το συνολικό κόστος και ένα πεδίο String το οποίο θα χρησιμοποιούμε για να τυπώνουμε κατάλληλα μηνύματα στον πελάτη.

BookingResult.java:

```
import java.io.Serializable;

public class BookingResult implements Serializable{
    private static final long serialVersionUID= 1L;

    public boolean success;
    public boolean partialAvailable;
    public int available;
    public int booked;
    public int totalCost;
    public String message;

    public BookingResult(boolean success, boolean partialAvailable, int available,
int booked, int totalCost, String message){
        this.success= success;
        this.partialAvailable= partialAvailable;
        this.available= available;
        this.booked= booked;
        this.totalCost= totalCost;
        this.message= message;
    }
}
```

Remote Interface:

Το επόμενο βήμα είναι η δημιουργία του interface δηλαδή να ορίσουμε τις συναρτήσεις που θα καλούν οι πελάτες από την γραμμή εντολών, σύμφωνα με την περιγραφή της άσκησης.

HRInterface.java:

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface HRInterface extends Remote{
    String listRooms() throws RemoteException;
    BookingResult book(String type, int number, String name, boolean allowPartial)
throws RemoteException;
    String listGuests() throws RemoteException;
```

```
String cancel(String type, int number, String name) throws RemoteException;
}
```

HRImplementation & HRClient:

Προχωράμε στην υλοποίηση της λογικής του προγράμματος μας που θα εκτελεί τις ζητούμενες λειτουργίες στο **HRImplementation.java**, και την υλοποίηση του client στο **HRClient.java**

Στο **implementation** ξεκινάμε αποθηκεύοντας τα δεδομένα του ξενοδοχείου, διαθέσιμα δωμάτια και τιμές σε δύο Hash Maps με κλειδιά τον τύπο των δωματίων. Αφού εκτελέσουμε έλεγχο για την εγκυρότητα του τύπου και σιγουρευτούμε ότι θα είναι σε κεφαλαία μορφή, ξεκινάμε την υλοποίηση των συναρτήσεων που ορίσαμε στο interface.

Από τα δύο αυτά αρχεία, θα σταθούμε εδώ στον τρόπο διαχείρισης της μερικής κράτησης δωματίων, όταν δηλαδή τα διαθέσιμα δωμάτια δεν επαρκούν και ο πελάτης επιθυμεί να κάνει κράτηση μόνο για όσα είναι διαθέσιμα.

Στην υλοποίηση του **client** καλούμε την συνάρτηση book για κράτηση δωματίων με το Boolean **allowPartial** όρισμα σε τιμή **false**.

```
HRInterface hotel= getRemoteObject(hostname);
BookingResult result= hotel.book(type, number, name, false);
System.out.println(result.message);
```

Το **implementation** με την σειρά του ελέγχει την **allowPartial** και αν είναι **false** δεν κάνει κράτηση αλλά επιστρέφει στον client την μεταβλητή **partialAvailable** σε **true** μαζί με σχετικό μήνυμα. Αν δεν υπάρχουν καθόλου δωμάτια η **partialAvailable** επιστρέφεται ως **false**.

Έπειτα ο **client** θα ελέγξει τις **success** & **partialAvailable** για να δει αν έγινε κράτηση και αν είναι δυνατή η μερική κράτηση. Αν δεν έγινε και είναι δυνατή, θα ρωτήσει τον πελάτη και αν είναι επιθυμητή η μερική κράτηση, ξανακαλείται η **book** αλλά με την **allowPartial** σε **true**.

HRClient.java

```
if(!result.success && result.partialAvailable){
    try (Scanner scanner = new Scanner(System.in)) {
        System.out.println("Would you like to book all available? (Y/N): ");
        String ans= scanner.nextLine();

        if(ans.equalsIgnoreCase("Y") || ans.equalsIgnoreCase("yes")){
            BookingResult partialResult= hotel.book(type, result.available, name,
true);
            System.out.println(partialResult.message);
        }
    }
}
```

Πίσω στο **implementation**, θα δει ότι η **allowPartial** είναι **true** και θα προχωρήσει στην πραγματοποίηση της μερικής κράτησης.

HRImpl.java

```
if(available >= number)
    roomsToBook = number;
else if(available > 0 && allowPartial)
    roomsToBook = available;
else if(available > 0)
    return new BookingResult(false, true, available, 0, 0, "Only " + available + "
type " + type + " room(s) available.");
else
    return new BookingResult(false, false, 0, 0, 0, "Booking error: No available
rooms.");

availableRooms.put(type, available - roomsToBook);
```

Επίσης, είναι σημαντικό να αναφέρουμε πως οι συναρτήσεις που υλοποιούνται στο **HRImpl.java** ορίζονται ως **synchronized**. Έτσι σιγουρευόμαστε πως μόνο ένας client την φορά μπαίνει σε κάποια συνάρτηση. Τέλος, για την υλοποίηση του remote interface, είναι απαραίτητο η κλάση (**HRImpl**) να κάνει extend την **UnicastRemoteObject**.

HRServer.java:

Με αυτό το αρχείο ξεκινάμε τον RMI server, δημιουργούμε ένα αντικείμενο **HRImpl** και το καταχωρούμε στο RMI registry ώστε να μπορούν να το βρίσκουν οι clients. Το αντικείμενο αυτό περιέχει όλη τη λογική του προγράμματος (διαθέσιμα δωμάτια, κρατήσεις, ακυρώσεις, λίστα πελατών) και θα χρησιμοποιείται κανονικά από τους clients, οι οποίοι θα το κάνουν lookup με όνομα "HotelReservation".

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class HRServer{
    public static void main(String[] args){
        try{
            HRImpl hotel = new HRImpl();
            Registry registry = LocateRegistry.createRegistry(3338);
            registry.rebind("HotelReservation", hotel);

            System.out.println("HR RMI server initialized.");
        }
        catch(Exception e){
            System.out.println("Server error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Πηγαίνοντας πίσω στο **HRClient.java**, έχουμε υλοποιήσει το lookup του remote object στην συνάρτηση

```
private static HRInterface getRemoteObject(String hostname) throws Exception{
    Registry registry= LocateRegistry.getRegistry(hostname, 3338);
    return (HRInterface) registry.lookup("HotelReservation");
}
```

και την καλούμε σε αρκετά σημεία του client με τον εξής τρόπο:

```
String hostname= args[1];
HRInterface hotel= getRemoteObject(hostname);
System.out.println(hotel.listGuests());
```

Στο συγκεκριμένο παράδειγμα, παίρνουμε το remote object για να δούμε την λίστα με τους πελάτες και τις κρατήσεις.

Ενδεικτικά Τρεξίματα:

1) Εκκίνηση server & Δοκιμή χωρίς παραμέτρους

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRServer
HR RMI server initialized.
```

```
● PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient
Available commands:
java HRClient list <hostname>
java HRClient book <hostname> <type> <number> <name>
java HRClient guests <hostname>
java HRClient cancel <hostname> <type> <number> <name>
```

2) Δοκιμή list

```
● PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient list localhost
Available rooms:
60 room(s) of type A - price: 90 Euros/night
50 room(s) of type B - price: 120 Euros/night
40 room(s) of type C - price: 150 Euros/night
30 room(s) of type D - price: 180 Euros/night
20 room(s) of type E - price: 225 Euros/night
```

3) Δοκιμή επιτυχούς κράτησης

```
● PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient book localhost A 2 Nikos
Booking successful: 2 room(s) of type A
Total cost: 180 Euros
```

```
● PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient list localhost
Available rooms:
58 room(s) of type A - price: 90 Euros/night
50 room(s) of type B - price: 120 Euros/night
40 room(s) of type C - price: 150 Euros/night
30 room(s) of type D - price: 180 Euros/night
20 room(s) of type E - price: 225 Euros/night
```

4) Δοκιμή guests

```
● PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient guests localhost
Customers & Reservations:
Nikos:
  2 room(s) of type A
  Cost: 180 Euros
  Total reservation cost: 180 Euros
```

5) Δοκιμή δεύτερης κράτησης στον ίδιο πελάτη

```
● PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient book localhost C 1 Nikos
Booking successful: 1 room(s) of type C
Total cost: 150 Euros
```

```
● PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient guests localhost
Customers & Reservations:
Nikos:
  2 room(s) of type A
  Cost: 180 Euros
  1 room(s) of type C
  Cost: 150 Euros
  Total reservation cost: 330 Euros
```

6) Δοκιμή κράτησης άλλου πελάτη

```
● PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient book localhost B 3 Maria
Booking successful: 3 room(s) of type B
Total cost: 360 Euros
```

```
● PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient guests localhost
Customers & Reservations:
Nikos:
  2 room(s) of type A
  Cost: 180 Euros
  1 room(s) of type C
  Cost: 150 Euros
  Total reservation cost: 330 Euros
Maria:
  3 room(s) of type B
  Cost: 360 Euros
  Total reservation cost: 360 Euros
```

7) Δοκιμή ακύρωσης

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient cancel localhost A 1 Nikos
Cancellation successful. Remaining client reservations:
A: 1 room(s).
C: 1 room(s).
Total cost: 240 Euros.
```

8) Δοκιμή αποτυχημένης ακύρωσης

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient cancel localhost A 10 Nikos
Cancellation error: Client has not enough reservations for room type A.
```

9) Δοκιμή μη αποδεκτού τύπου δωματίου

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient book localhost Z 1 John
Booking error: invalid room type.
```

10) Δοκιμή μερικής κράτησης

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient book localhost E 25 Alex
Only 20 type E room(s) available.
Would you like to book all available? (Y/N):
y
Booking successful: 20 room(s) of type E
Total cost: 4500 Euros
```

11) Δοκιμή αποτυχίας όταν δεν υπάρχουν καθόλου δωμάτια

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient list localhost
Available rooms:
59 room(s) of type A - price: 90 Euros/night
47 room(s) of type B - price: 120 Euros/night
39 room(s) of type C - price: 150 Euros/night
30 room(s) of type D - price: 180 Euros/night
0 room(s) of type E - price: 225 Euros/night
```

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient book localhost E 1 Ioanna
Booking error: No available rooms.
```

Μέρος Β:

Για να υλοποιήσουμε την λειτουργία των ειδοποιήσεων πρέπει ο client να δώσει στον server ένα remote object, μέσω του οποίου θα μπορεί ο server να καλεί την λειτουργία αυτή πάνω στον client. Θα κάνουμε δηλαδή το αντίστροφο απ' ότι κάναμε μέχρι τώρα.

Callback Interface:

Ξεκινάμε δημιουργώντας το interface της συνάρτησης που θα καλεί ο server όταν θα θέλει να ειδοποιήσει έναν client

ClientCallback.java:

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface ClientCallback extends Remote{
    void messageClient(String message) throws RemoteException;
}
```

και προχωράμε στην υλοποίηση της δημιουργώντας το

ClientCallbackImpl.java:

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class ClientCallbackImpl extends UnicastRemoteObject implements
ClientCallback{
    public ClientCallbackImpl() throws RemoteException{
        super();
    }

    @Override
    public void messageClient(String message) throws RemoteException{
        System.out.println("\n+++ RESERVATION NOTIFICATION +++");
        System.out.println(message);
        System.out.println("Exit and try again.");
    }
}
```

Έπειτα προσθέτουμε την συνάρτηση **registerForNotification()** στο **HRInterface.java**:

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface HRInterface extends Remote{
    [.....]
    void registerForNotification(String type, String name, ClientCallback callback)
throws RemoteException;
}
```


Δηλαδή ο client θα καλεί

```
hotel.registerForNotification(type, name, callback);
```

και ο server θα αποθηκεύει τον client σε λίστα ειδοποιήσεων για τον συγκεκριμένο τύπο δωματίου και θα τυπώνει τα απαραίτητα μηνύματα, όπως θα δούμε παρακάτω.

HRImpL.java:

Τώρα πρέπει να επεξεργαστούμε το **HRImpL.java**:

Ορίζουμε τις λίστες ειδοποίησης σε μορφή **HashMap** με κλειδί τον **τύπο** δωματίου και τις αρχικοποιούμε στον constructor:

```
private final Map<String, Map<String, ClientCallback>> notificationLists;

public HRImpL() throws RemoteException{
    super();

    availableRooms= new HashMap<>();
    prices= new HashMap<>();
    reservations= new HashMap<>();
    notificationLists= new HashMap<>();

    availableRooms.put("A", 60);
    [...]
    prices.put("E", 225);

    notificationLists.put("A", new HashMap<>());
    notificationLists.put("B", new HashMap<>());
    notificationLists.put("C", new HashMap<>());
    notificationLists.put("D", new HashMap<>());
    notificationLists.put("E", new HashMap<>());
}
```

Και υλοποιούμε την συνάρτηση **registerForNotification**:

```
@Override
public synchronized void registerForNotification(String type, String name,
ClientCallback callback) throws RemoteException{
    type= normalizeType(type);

    if(!isValidType(type))
        throw new RemoteException("Invalid room type");
    if(name== null || name.trim().isEmpty())
        throw new RemoteException("Client name is empty.");

    notificationLists.get(type).put(name, callback);
    System.out.println(name + " is now registered for notifications about room
type " + type);
}
```

Αν ο χρήστης αποτύχει να κάνει κράτηση και επιλέξει να μπει στην λίστα ειδοποιήσεων τότε ο server τον προσθέτει στο **notificationLists Map** και έτσι όταν ελευθερωθεί κάποιο δωμάτιο, ο server θα γνωρίζει ποιους να ειδοποιήσει.

Τώρα που μπορούμε να προσθέτουμε τους χρήστες στην λίστα, πρέπει να υλοποιήσουμε την αποστολή των ειδοποιήσεων. Προσθέτουμε λοιπόν και την συνάρτηση **notifyClients()** στο **HRImpl**:

```
private void notifyClients(String type){
    Map<String, ClientCallback> clients= notificationLists.get(type);
    if(clients== null | clients.isEmpty())
        return;

    String message= "" + availableRooms.get(type) + " room(s) of type " + type + "
are now available.";

    for(Map.Entry<String, ClientCallback> entry : clients.entrySet()){
        try{
            entry.getValue().messageClient(message);
        }
        catch(RemoteException e){
            System.out.println("Could not notify client " + entry.getKey());
        }
    }

    clients.clear();
}
```

και την καλούμε μέσα στο **cancel** μετά από κάθε επιτυχημένη ακύρωση:

```
171: availableRooms.put(type, availableRooms.get(type) + number);
172: notifyClients(type);
```

Έτσι για κάθε ακύρωση δωματίων, στέλνουμε ειδοποίηση στους clients που ενδέχεται να περιμένουν για αυτόν τον τύπο δωματίου.

HRClient.java:

Οι τελευταίες αλλαγές που πρέπει να κάνουμε είναι στο **HRClient.java**:

Θα προσθέσουμε την λογική στο branch του **book** (ln: 34), μετά το partial booking.

Συγκεκριμένα, προσθέτουμε την Boolean

```
boolean bookingCompleted = result.success;
```

η οποία αποθηκεύει αν τελικά έγινε κάποια κράτηση. Αν δεν έγινε, είτε δεν υπήρχαν καθόλου διαθέσιμα δωμάτια είτε ο πελάτης δεν επέλεξε μερική κράτηση, τον ρωτάμε αν επιθυμεί να μπει στην λίστα ειδοποιήσεων και αν απαντήσει θετικά καλούμε την **registerForNotification()**.

Θα πρέπει όμως το πρόγραμμα του client να αναμένει την ειδοποίηση και συνεπώς θα πρέπει να παραμείνει ανοιχτό για κάποιο χρονικό διάστημα:

```
while(true)
    Thread.sleep(1000);
```

Συνολικά οι αλλαγές μέσα στο **HRClient** είναι:

```
[.....]
    if(ans.equalsIgnoreCase("y") || ans.equalsIgnoreCase("yes")){
        BookingResult partialResult= hotel.book(type, result.available, name,
true);
        System.out.println(partialResult.message);

        bookingCompleted= partialResult.success;
    }
}

if(!bookingCompleted){
    try (Scanner scanner = new Scanner(System.in)) {
        System.out.println("Would you like to be notified when room type " + type + "
is available? (Y/N): ");
        String ans= scanner.nextLine();

        if(ans.equalsIgnoreCase("y") || ans.equalsIgnoreCase("yes")){
            ClientCallback callback= new ClientCallbackImpl();
            hotel.registerForNotification(type, name, callback);

            System.out.println("Registration successful.");
            System.out.println("Keep this window open to receive notifications.");

            while(true)
                Thread.sleep(1000);
        }
    }
}
```

Δοκιμή:

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient book localhost E 20 George
Booking successful: 20 room(s) of type E
Total cost: 4500 Euros
```

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient book localhost E 1 Maria
Booking error: No available rooms.
Would you like to be notified when room type E is available? (Y/N):
y
Registration successful.
Keep this window open to receive notifications.
```

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient cancel localhost E 1 George
Cancellation successful. Remaining client reservations:
E: 19 room(s).
Total cost: 4275 Euros.
```

```
PS C:\Users\Rober\Desktop\ask2_23390338\hotelReservations\src> java HRClient book localhost E 1 Maria
Booking error: No available rooms.
Would you like to be notified when room type E is available? (Y/N):
y
Registration successful.
Keep this window open to receive notifications.

+++ RESERVATION NOTIFICATION +++
1 room(s) of type E are now available.
Exit and try again.
```

Μέρος Γ:

Πως θα επεκτείνετε την εφαρμογή σας ώστε,

A) Να υποστηρίζει κρατήσεις για πολλά ξενοδοχεία και ημ. άφιξης/αναχώρησης

Για να μπορέσουμε να υποστηρίξουμε πολλά ξενοδοχεία, θα πρέπει να δημιουργήσουμε ξεχωριστή οντότητα **Hotel**. Κάθε ξενοδοχείο θα έχει δικό του αναγνωριστικό, όνομα, σύνολο διαθέσιμων δωματίων και τύπους δωματίων. Συνεπώς οι διάφορες συναρτήσεις μας θα πρέπει να αλλάξουν για να δέχονται τα επιπλέον αυτά δεδομένα (ορίσματα) και να τα διαχειρίζονται π.χ. οι **book & listRooms**.

Για την υποστήριξη ημερομηνιών, η διαθεσιμότητα δεν θα μπορεί να είναι πλέον απλός ένας αριθμός γιατί θα πρέπει να την ελέγχουμε με βάση την επιθυμητή ημερομηνία του πελάτη. Τα δωμάτια μπορεί να μην είναι διαθέσιμα για κάποιες μέρες και να είναι για άλλες.

B) Να μπορεί να χρησιμοποιηθεί διαδικτυακά

Ο χρήστης δεν θα τρέχει πλέον την εφαρμογή από την κονσόλα αλλά μέσα από μία ιστοσελίδα. Η web εφαρμογή θα μπορούσε να υλοποιηθεί με web frameworks όπως Java Servlets, JSP ή Spring Boot. Το RMI θα μπορούσαμε να το αφήσουμε ως εσωτερικό μηχανισμό επικοινωνίας μεταξύ web app και HR server ή να το αντικαθιστούσαμε με REST API ή HTML φόρμες.

Γ) Να συνεργάζεται με RDBMS για την οργάνωση-απόθηκευση δεδομένων

Θα χρειαστεί να μεταφέρουμε τα δεδομένα από τα HashMaps σε μία βάση δεδομένων. Θα δημιουργήσουμε πίνακες όπως HOTEL, ROOM_TYPE, ROOM_AVAIL, κτλ. και έτσι έχουμε καλύτερη, ασφαλέστερη και μόνιμη αποθήκευση των δεδομένων μας, ακόμη και αν κλείσουμε τον HR server.

Η αναζήτηση και ενημέρωση των δεδομένων θα γίνεται από την εφαρμογή μας μέσω JDBC και οι συναρτήσεις μας θα πρέπει να ενημερώνουν την βάση αντί για τα HashMaps, εκτελώντας SQL queries.